

Московский государственный технический университет

Факультет ИУ «Информатика и системы управления»

Кафедра ИУ-1 «Системы автоматического управления»

ОТЧЕТ

по домашней работе №2

по дисциплине

«Микропроцессорные и встраиваемые системы»

Выполнил: Салженикин Д.А

Группа: ИУ1-52Б

Проверил: Ермаков И.Ю.

Работа выполнена: 22.12.2023

Отчет сдан:

Оценка:

2023 г.

Задание

Разработать программу, реализующую установку напряжения на внешнем ЦАП DAC7611 согласно документации на микросхему. Значение напряжения задаётся посредством модуля USART2. Формат команды: «U u.uuu», где u.uuu напряжение на выходе ЦАП в Вольтах.

Решение

Решение будет заключаться в том, что сначала посредством модуля USART получаем число формата «U u.uuu», а затем через SPI передаем данные на ADC. Для измерения будем использовать вольтметр

Блок – схема проиллюстрирована на рисунке 1.



Рисунок 1 – Блок-схема

Для тестирования написанной программы используется программа симуляции Proteus v8.13. Электрическая схема проиллюстрирована на рисунке 2.

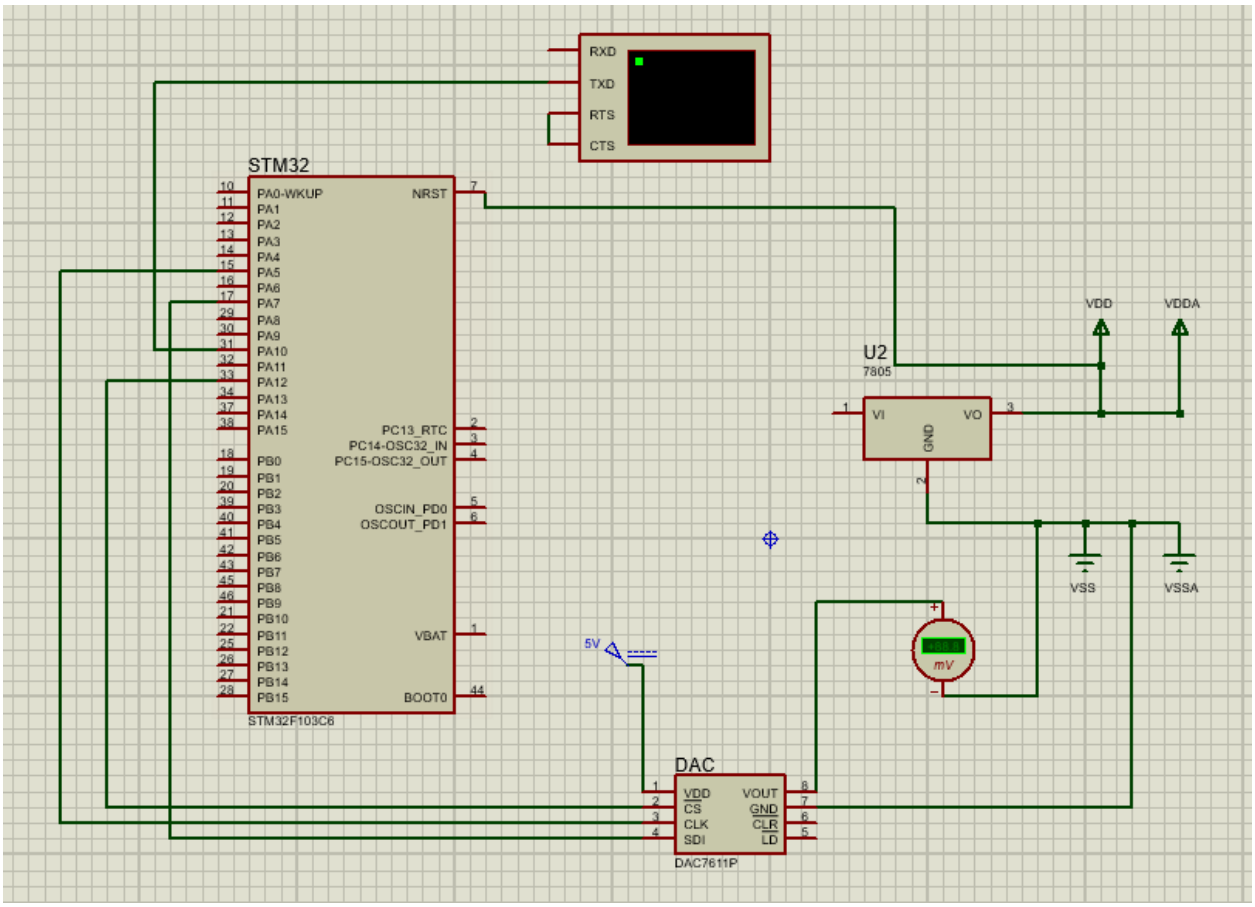


Рисунок 2 – Электрическая схема

Согласно блок-схеме алгоритма была разработана программа для микроконтроллера STM32F103C8T6 на Си с использованием библиотеки CMSIS и среды разработки STM32CubeIDE. Текст разработанной программы содержится в Листинге 1.

Листинг 1.

```
/* USER CODE BEGIN Header */
/**

******************************************************************************
*
  * @file           : main.c
  * @brief          : Main program body
```

```
******************************************************************************
*
  * @attention
  *
  * Copyright (c) 2023 STMicroelectronics.
  * All rights reserved.
  *
  * This software is licensed under terms that can be found in the LICENSE
file
  * in the root directory of this software component.
  * If no LICENSE file comes with this software, it is provided AS-IS.
  *

******************************************************************************
*
  */
/* USER CODE END Header */
/* Includes ------------------------------------------------------------------
-*/

#include "main.h"
#include "stm32f1xx.h"
#include <stdio.h>
/* Private includes ----------------------------------------------------------
-*/
/* USER CODE BEGIN Includes */

/* USER CODE END Includes */

/* Private typedef -----------------------------------------------------------
-*/
/* USER CODE BEGIN PTD */

/* USER CODE END PTD */

/* Private define ------------------------------------------------------------
-*/
/* USER CODE BEGIN PD */

/* USER CODE END PD */

/* Private macro -------------------------------------------------------------
-*/
/* USER CODE BEGIN PM */

/* USER CODE END PM */

/* Private variables ---------------------------------------------------------
-*/

/* USER CODE BEGIN PV */
uint32_t tiks = 0;
/* USER CODE END PV */

/* Private function prototypes -----------------------------------------------
-*/
/* USER CODE BEGIN PFP */

void SysTickIRQHandler(void);
void delayMS(uint32_t delay);
```

```
void Configure_USART2(void);
void Configure_DAC7611(void);
void Configure_GPIO(void);

void SPI_Init(void);

void DAC_SetVoltage(float voltage);

/* USER CODE END PFP */

/* Private user code ---------------------------------------------------------
-*/
/* USER CODE BEGIN 0 */
void SysTickIRQHandler(void){
    tiks++;
}

void delayMS(uint32_t delay){
    uint32_t currentTiks = tiks;
    while(tiks - currentTiks < delay);
}

void Configure_USART1(void) {
    // Включение тактирования USART1
    RCC->APB2ENR |= RCC_APB2ENR_USART1EN; //PA9 and PA10

    USART1->BRR = 0x1A0B; // Для скорости 9600 бит/с при частоте системной
шины 64 МГц

    // Настройка параметров асинхронного режима: 8 бит данных, 1 стоп-бит,
без контроля четности
    USART1->CR1 &= ~(USART_CR1_M); //8 бит данных
    USART1->CR2 &= ~(USART_CR2_STOP); // 1 стоп-бит
    USART1->CR1 &= ~(USART_CR1_PCE | USART_CR1_PS); //бит четности отключен

    // Включение USART1
    USART1->CR1 |= USART_CR1_UE; //выставили в 1 - это позволит USART
отправлять и принимать данные.
    USART1->CR1 |= USART_CR1_TE | USART_CR1_RE;
}

void Configure_GPIO(void) {

    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;  // Включение тактирования порта A

    GPIOA->CRH &= ~(GPIO_CRH_CNF9 | GPIO_CRH_CNF10 | GPIO_CRH_MODE9 |
GPIO_CRH_MODE10);
    GPIOA->CRH |= GPIO_CRH_CNF9_1 | GPIO_CRH_MODE9_0; // TX (PA9) - AF, 50MHz
    GPIOA->CRH |= GPIO_CRH_CNF10_0; // RX (PA10) - Floating input
}

void SPI_Init(void) {
    RCC->APB2ENR |= RCC_APB2ENR_SPI1EN; // Включаем тактирование SPI1
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN; // Включаем тактирование порта A

    // Настраиваем пины SPI1 (PA5 - SCK, PA7 - MOSI)
    GPIOA->CRL |= GPIO_CRL_MODE5_1 | GPIO_CRL_CNF5_1; // Режим
"альтернативная функция, push-pull", скорость 2 МГц
    GPIOA->CRL |= GPIO_CRL_MODE7_1 | GPIO_CRL_CNF7_1; // Режим
"альтернативная функция, push-pull", скорость 2 МГц
```

```
    // Настраиваем пины CS
    GPIOA->CRH &= ~(GPIO_CRH_MODE12);
    GPIOA->CRH |= GPIO_CRH_CNF12_0; // CS - Floating input

    // Настраиваем настройки SPI1
    SPI1->CR1 = SPI_CR1_MSTR | SPI_CR1_BR_2 | SPI_CR1_DFF; // Мастер, f_pclk
/ 16 (настройка скорости передачи), 8 бит на передачу данных
    SPI1->CR1 |= SPI_CR1_SPE; // Включаем SPI
}

void DAC_SetVoltage(float voltage) {
    uint16_t dac_data = (uint16_t)(4095 * voltage / 3.3); // Преобразуем
напряжение в данные ЦАП

    // Ждем, пока SPI1 готов к передаче данных
    while (!(SPI1->SR & SPI_SR_TXE));

    // Записываем данные в буфер передачи
    SPI1->DR = (dac_data >> 8) & 0xFF;

    while (!(SPI1->SR & SPI_SR_TXE));
    SPI1->DR = dac_data & 0xFF;

    // Ждем, пока SPI1 завершит передачу
    while (SPI1->SR & SPI_SR_BSY);
}



/* USER CODE END 0 */

/**
  * @brief  The application entry point.
  * @retval int
  */
int main(void)
{
  /* USER CODE BEGIN 1 */

  /* USER CODE END 1 */

  /* MCU Configuration--------------------------------------------------------
-*/

  /* Reset of all peripherals, Initializes the Flash interface and the
Systick. */


  /* USER CODE BEGIN Init */

  /* USER CODE END Init */

  /* Configure the system clock */

  /* USER CODE BEGIN SysInit */

  /* USER CODE END SysInit */

  /* Initialize all configured peripherals */
  /* USER CODE BEGIN 2 */
```

```
    RCC->CFGR |= 0xF << RCC_CFGR_PLLMULL_Pos; // частота 64 МГц
    RCC->CR |= RCC_CR_PLLON;
    while(!(RCC->CR & RCC_CR_PLLRDY));
    RCC->CFGR |= 2 << RCC_CFGR_SW_Pos;
    while(!(RCC->CFGR & (2 << RCC_CFGR_SWS_Pos))); //частота настроена на 64
мГц
    SysTick_Config(64000000/1000);
    NVIC_EnableIRQ(SysTick_IRQn);

    Configure_GPIO();
    Configure_USART2();
    SPI_Init();
    /* USER CODE END 2 */

  /* Infinite loop */
  /* USER CODE BEGIN WHILE */
  while (1)
  {

    if (USART1->SR & USART_SR_RXNE) {
        // Принимаем символ из USART1
        char receivedChar = USART1->DR;

        // Обработка символа
        if (receivedChar == 'U') {
            // Обработка команды установки напряжения

            char buffer[10];
            // Принимаем строку, содержащую напряжение
            scanf("U %s", buffer);

            float voltage;
            // Преобразуем строку во float
            sscanf(buffer, "%f", &voltage);

            // Отправляем символ конца строки
            USART1->DR = 0x00;
            // Вызываем функцию для установки напряжения на ЦАП
            DAC_SetVoltage(voltage);
        }
    delayMS(1000);
    }

    /* USER CODE END WHILE */

    /* USER CODE BEGIN 3 */
  }
  /* USER CODE END 3 */
}
```

Для проверки будем использовать вольтметр к “+” которого подключим выход напряжения ЦАП DAC7611. А “-” заземлим с общим выводом GND. Результат моделирования показан на рисунке 3.

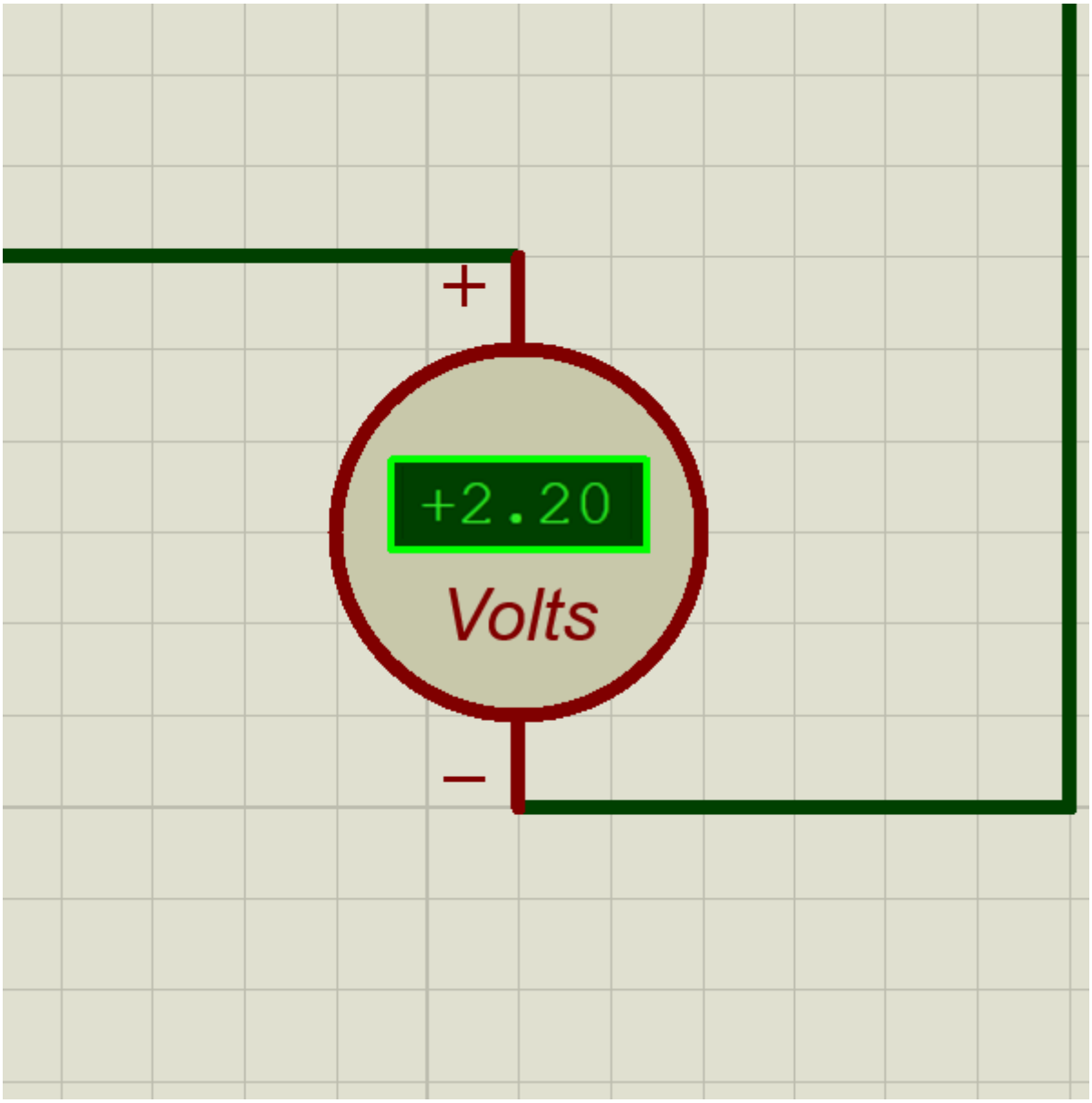


Рисунок 3 – Вывод напряжения с DAC7611

Вывод

Был разработан алгоритм и написана программа для микроконтроллера STM32F103C8T6 на языке Си с использованием библиотеки CMSIS, разработана схема принципиальная, проверена ее работа в симуляторе Proteus.